

Project title: Appointment Organizer (APO)

Contents

Introduction	2
Scope.....	2
Concept	3
Software structure	4
Testing cases.....	5
Performance analysis	6
Program set up and execution.....	6
Contribution of individual member	错误!未定义书签。
Appendix	8

Introduction

The objective of this project is to apply the theoretical knowledge of process scheduling learned in COMP2432 to a real-world scenario and develop a software application that can help users manage their appointments efficiently.

The project aims to create an Appointment Organizer (APO) that allows users to add details of appointments, check the availability of all parties involved, schedule appointments using several scheduling algorithms, and generate a timetable for all appointments including rejected appointments.

APO only can execute with at least three users and at most ten users.

APO will help users schedule the time of 18:00 to 23:00 without the Sunday and public holidays. Users can input the details of private time, project meeting, group study, and gathering with their date, starting time, and duration. The project meeting, group study, and gathering also can invite other users.

Users can end the program when they have finished using APO.

Scope

This project covered operating systems topics of below:

- System Calls
 - APO needs to be run and read on Unix using system calls.
- Process Management
 - CPU scheduling
 - ◆ APO scheduler uses scheduling algorithms (FCFS, Priority) for the involved parties' sequence of appointments.
 - APO uses Fork to create child processes.
- Interprocess communication and programming
 - APO is Asynchronous for I/O processing.
 - Interprocess communication (IPC)
 - ◆ Message passing
 - Parent to Children communications; Unnamed Pipe.
 - ◆ Asynchronous for I/O processing: Parent and children are separate processes until they communicate.
- File Management
 - APO will read and write the file.

Concept

APO uses FCFS and Priority scheduling algorithms to print the appointment timetable for users.

First Come First Serve(FCFS)

The timetable priority scheduling is the order arrival time.

e.g., First, mary order a group study with john from 18:00 to 20:00 on May 1. Second, mary order a private time from 18:00 to 20:00 on May 1. Third, mary order a gathering with lucy from 21:00 to 22:00 on May 1.

In this case, private time will be rejected, the chronology timetable of mary is 18:00 to 20:00 group study ,21:00 to 22:00 gathering on May 1.

Priority Scheduling

The timetable priority scheduling follows the below:

1. Private time
2. Project meeting
3. Group study
4. Gathering

Communication via Pipes

In Unix/Linux, processes could use the pipe for communication. A pipe is the simplest form of Inter-Process Communication (IPC) mechanism in Unix/Linux, and is a message-based direct communication mechanism. A file is represented by a file descriptor and a pipe is represented by a pair of file descriptors; the first one is for reading and the second one is for writing. A pipe is created by the system call `pipe()`. An array of integer of size 2 is passed as an argument for the system call to return the pair of file descriptors.

The `pipe()` system call will accept an array of 2 file descriptors (integers) and create a data structure in the kernel space, as illustrated in the diagram. You could consider the pipe structure to be a queue.

One can insert elements to the end of the queue by writing to the pipe, and remove elements from the front of the queue by reading from the pipe.

File I/O Library Calls

As long as Linux and Unix are concerned, the file is a very important concept. Virtually everything is considered to be a file in Unix/Linux. To use a file, we need to define a pointer to a file of data type `FILE`. This pointer of type `*FILE` is called a file

handle. To open a file for read/write, use fopen library call. To close a file, use fclose library call. Note that a library call is a higher level mechanism than a system call, providing a more user-friendly interface for a programmer.

To open a file for reading, use “r” (meaning read); use “w” (meaning write); A return value of NULL means an error in both library calls. You could read input from a file using fscanf similar to scanf. You could send output to a file using fprintf similar to printf. There is a special file handle, called stderr, to be used when printing an error message. By default, it will go to screen unless being mapped to elsewhere (e.g., a file). It is a common practice in C programs that one would use stderr for errors.

QuickSort

QuickSortLike Merge Sort, QuickSort is a Divide and Conquer algorithm. It picks an element as a pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

Always pick the first element as a pivot.

Always pick the last element as a pivot (implemented below)

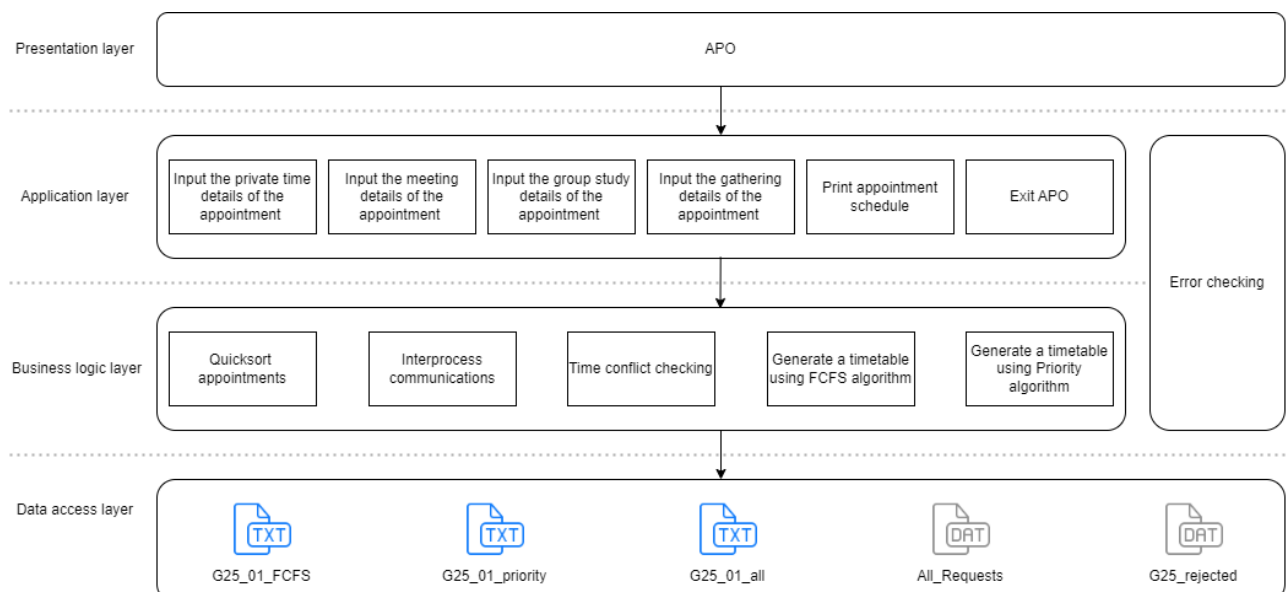
Pick a random element as a pivot.

Pick median as the pivot.

The key process in quickSort is a partition(). The target of partitions is, given an array and an element x of an array as the pivot, put x at its correct position in a sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x. All this should be done in linear time.

Software structure

APO is running on the apollo, a server machine with a CentOS Linux operating system installed. APO is the layered pattern structure.



Testing cases

This project is using **printf** to print out the details of interprocess communication with the user, PID, number, size, and length of the received message to check if the processes and the code function properly.

printf whenever enter a command such as privateTime paul 20230401 1800 2.0

Successful Test Examples:

```
~~~WELCOME TO APO~~~
# users:
children, pid 231 ,No 1: User john create successfully
children, pid 232 ,No 2: User mary create successfully
children, pid 233 ,No 3: User lucy create successfully
children, pid 234 ,No 4: User paul create successfully
Please enter appointment:

Please enter appointment:
children, pid 5034 ,No 3: User lucy create successfully
privateTime paul 20230401 1800 2.0
→ [Recorded]
<parent> sending message [privateTime paul 20230401 1800 2.0] to child
children, pid 5035 ,No 4: User paul message [privateTime] callee [paul] of size 304 bytes received
<parent> received children 4 message [get] is of length 3
Please enter appointment:

Please enter appointment:
printSchd FCFS
children, pid 5035 ,No 4: User paul message [privateTime] caller [paul] of size 304 bytes received
<parent> received children 4 message [true] is of length 4
<parent> received children 4 message [success] is of length 7
1 : write file
<parent> received children 1 message is of length 8
2 : write file
<parent> received children 2 message is of length 8
3 : write file
<parent> received children 3 message is of length 8
4 : write file
<parent> received children 4 message is of length 8
→ [Exported file: G25_01_FCFS.txt] Please enter appointment:
```

Command and Syntax Usage

1. To execute the program - ./apo YYYYMMDD YYYYMMDD u1 u2 u3 ...
2. Bash Input - fileInput fileName
Filename could be G25_tests.dat or All_Requests.dat
3. privateTime - privateTime uuu YYYYMMDD hhmm n.n
Bash input - privateTime paul 20230401 1800 2.0
4. projectMeeting - projectMeeting uuu YYYY-MM-DD hh:mm n.n u1 u2 ...
Bash input - projectMeeting john 20230402 1900 2.0 paul mary
5. groupStudy - groupStudy uuu YYYYMMDD hhmm n.n u1 u2 ...
Bash input - groupStudy paul 20230403 1800 2.0 john lucy
6. Gathering - gathering uuu YYYYMMDD hhmm n.n u1 u2 ...
Bash input - gathering lucy 20230404 1900 4.0 john paul mary
7. printSchd - printSchd sssss / ALL

[sssss] – Scheduling algorithm (FCFS/ PRIORITY)

[ALL] – All scheduling algorithms are included in this report

Bash input - printSchd FCFS / printSchd PRIORITY / printSchd ALL

8.endProgram - endProgram

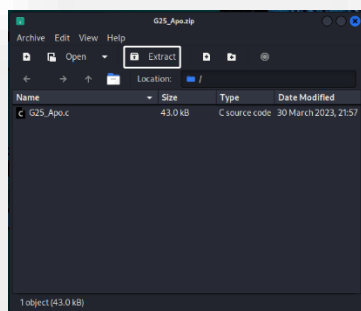
Performance analysis

APO used two scheduling algorithms: FCFS and Priority

	Advantages	Disadvantages
FCFS	The FCFS algorithm is fair and simple to implement.	There is no way to schedule based on priorities.
Priority	Higher priority appointments will be prioritized.	The algorithm is unfair. Appointments with higher priority will replace appointments that arrive first at the same time.

Program set up and execution

1. unzip the G25.zip file



2. put the unzipped file in the apollo with own address

3. login to the apollo and change the current directory to the file.

4. enter ***gcc G25_Apo.c -o G25_Apo -std=c99*** in apollo. PS: There might be many

warnings, just ignore them.

```
G25_Apo.c:39:27: warning: initialization of 'char' from 'void *' makes integer from pointer without a cast [-Wint-conversion]
  39 | struct fileInput input = {NULL, 0, 0};
      |                          ^~~~~
G25_Apo.c:39:27: note: (near initialization for 'input.fileLine[0][0]')
G25_Apo.c: In function 'main':
G25_Apo.c:1097:28: warning: implicit declaration of function 'wait' [-Wimplicit-function-declaration]
1097 |         if ((waitPid = wait(&status)) < 0) // The waitPid means the id num of children
      |                        ^~~~~
```

5. enter **`./G25_Apo YYYYMMDD [START DATE] YYYYMMDDD [END DATE] u1 u2 u3 [3-10 Names separated with space]`**

For example: **`./apo 20230401 20230430 john mary lucy paul`**

`./G25_Apo 20230401 20230430 john mary lucy paul`

Execute example

```
~~WELCOME TO APO~~
4 users:
Please enter appointment:
privateTime paul 20230401 1800 2.0
-> [Recorded]
Please enter appointment:
privateTime mary 20230402 2000 3.0
-> [Recorded]
Please enter appointment:
projectMeeting john 20230402 1900 2.0 paul mary
-> [Recorded]
Please enter appointment:
groupStudy paul 20230403 1800 2.0 john lucy
-> [Recorded]
Please enter appointment:
gathering lucy 20230404 1900 4.0 john paul mary
-> [Recorded]
Please enter appointment:
printSchd FCFS
-> [Exported file: G25_01_FCFS.txt]
Please enter appointment:
printSchd PRIORITY
-> [Exported file: G25_01_Priority.txt]
Please enter appointment:
projectMeeting john 20230429 2000 2.0 lucy mary
-> [Recorded]
Please enter appointment:
printSchd ALL
-> [Exported file: G25_01_ALL.txt]
Please enter appointment:
endProgram
-> Bye!
```

Also, user can use bash input to execute APO.

`fileInput [file name of the bash input]`

For example: **`fileInput G25_tests.dat`**

Appendix

G25_Apo.c is the program APO.

All_Requests.dat is the Log file of all input requests.

G25_rejected.dat is the Log file of rejected requests .

G25_01_Priority.txt is the samples of appointment schedules using Priority scheduling.

G25_01_FCFS.txt is the samples of appointment schedules using FCFS scheduling.

G25_01_ALL.txt is the samples of appointment schedules using FCFS and Priority scheduling.

G25_tests.dat records all inputs used in testing the program.